Codeer Agent Skill 客戶安裝指南

這份指南協助你完成 Codeer CLI 與 Codeer Agent Skill 安裝。

重點不是避開 Web UI，而是讓 Codex / Claude Code 能參與完整的 Codeer Agent 生命週期：從資料整理、Agent 規劃、Behavior Cases、Production History 分析，到下一輪迭代。

先分清楚：CLI 和 Skill 各自負責什麼？

Codeer CLI

Codeer CLI 是安裝在本機的 codeer 指令工具。

它負責：

- 登入 Codeer
- 連接 Codeer API
- 執行 codeer agent、codeer eval、codeer history、codeer kb 等指令
- 驗證 API key、workspace、agent 權限是否正常
- 使用 codeer skill 安裝、列出或移除 Codeer Skills（CLI 0.1.17 起）

Codeer Agent Skill

Codeer Agent Skill 是安裝到 Codex / Claude Code 的工作說明。

它負責讓 assistant 知道：

- Codeer Agent 生命週期怎麼走
- 如何從 KB / 檔案討論並建立 Agent v0
- 如何產生 Behavior Cases
- 如何分析 Production History / Negative Feedback
- 什麼時候要先讓你看會改哪些內容，再由你確認是否套用或發布

Codeer Agent Skill 可以發揮什麼威力

1. 從 KB / 檔案開始規劃並創建 Agent v0

把文件、FAQ、SOP、產品資料或資料夾交給 Codex / Claude Code，先討論 Agent 範圍、知識邊界與初版設計。

2. 產生 Behavior Cases

先選擇或設計 Tester、確認它能取得必要證據，再建立 Case、Rubric 與配對。 Tester 就是既有 Evaluator / Judge，讓品質討論有具體依據。

3. 根據失敗問題作出調整

失敗不一定代表 prompt 錯；也可能是 KB、Agent、retrieval、Standard / Rubric 需要修正。

4. 分析 Production History / Negative Feedback

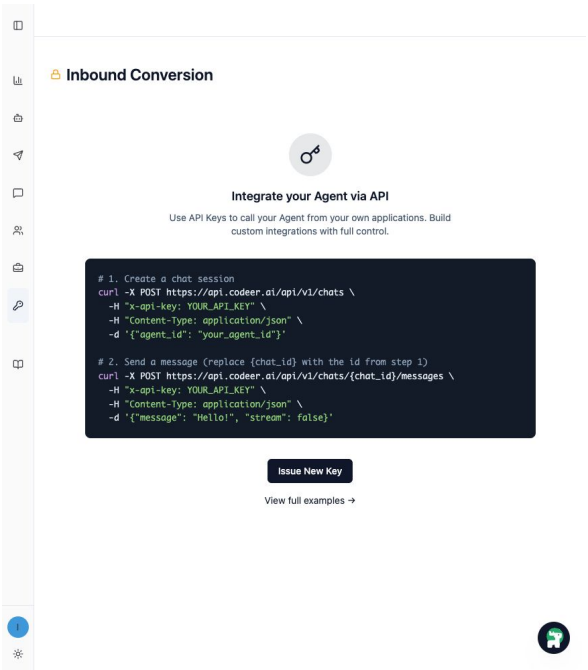
從真實對話和負評中找出模式、缺口與高風險場景，讓下一輪改善有證據。

5. 協助你開始下一輪的迭代

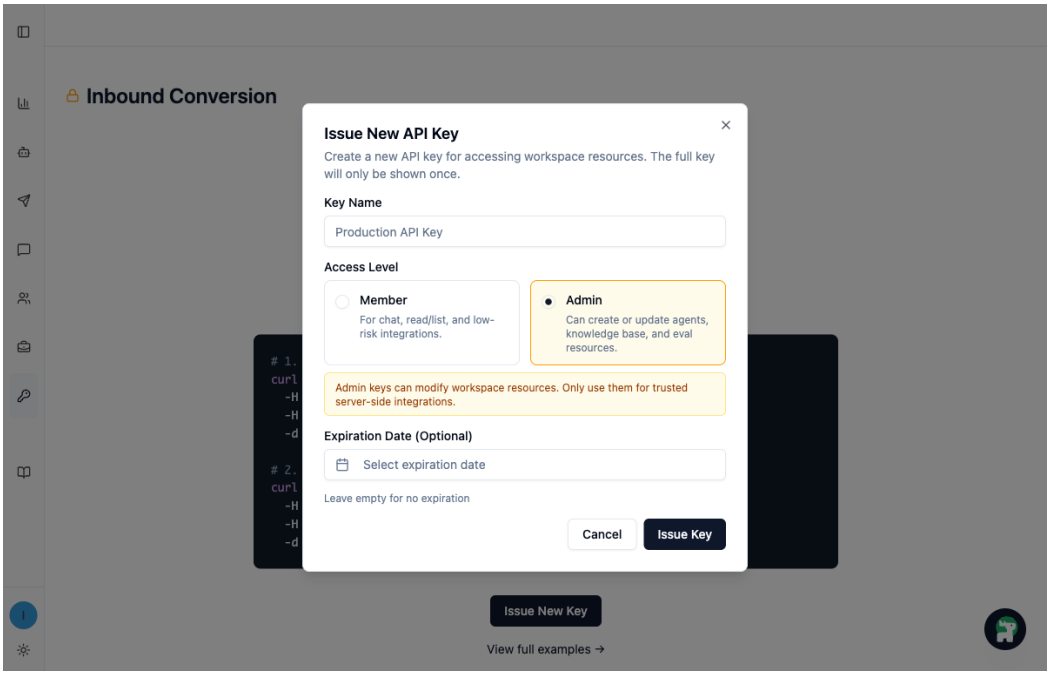
先整理問題、提出修正、預覽會改哪些內容，再由你確認是否套用或發布。

Step 1：在 Codeer 建立 API Key

請使用目標 workspace 的 Admin API key。Skill 會透過 CLI 做 Agent、 KB、Behavior Cases 和 History 相關工作，因此初次設定建議使用 Admin 權限。



在 workspace 頁面左側打開 API Keys，點選 Issue New Key。



輸入容易辨識的名稱，例如 Codex 或 Claude Code，Access level 選 Admin。

完整 API key 只會顯示一次。請立即存到安全位置，不要貼進聊天、prompt、 Git repo、專案 .env 或 .claude/settings.json。

Step 2：安裝 Codeer CLI

Codeer Agent Skill 會透過本機 codeer CLI 操作 Codeer API。建議使用 pipx 安裝，讓 CLI 跟專案 Python 環境分開，比較容易維護。

安裝 CLI：

```
pipx install codeer-cli
```

確認指令可用：

```
codeer --help
```

如果沒有 pipx：

```
python -m pip install --user pipx
python -m pipx ensurepath
```

重開 terminal 後，再執行：

```
pipx install codeer-cli
```

維護更新：

```
pipx upgrade codeer-cli
codeer check
```

不建議把 python -m pip install codeer-cli 當作主要安裝方式。它可以作為 fallback，但 pipx 較不容易受到專案套件版本影響。

Step 3：設定本機 CLI Profile

Profile 會把 API key 存在使用者層級的 Codeer CLI config，不需要放進 Codex 或 Claude Code 的設定檔。

下面的 work 只是範例 profile 名稱。你可以換成自己容易辨識的名字，例如 codeer、prod、client-a 或 support-agent。如果你改了名稱，後面的 codeer profile use ... 也要使用同一個名字。

新增 profile：

```
codeer profile add work
```

CLI 會請你貼上 API key，輸入時不會顯示在畫面上。

選用 profile：

```
codeer profile use work
```

檢查 CLI 與 API 是否可用：

```
codeer check
codeer agent list
```

這只能確認 CLI 和 API credential 正常，還不是 Skill 驗證。

如果專案只有一個預設 Agent，可以選填：

```
export CODEER_AGENT_ID=<agent-id>
```

不要把 CODEER_API_KEY 放進 .claude/settings.json、專案 .env、聊天訊息 或 Git commit。

Step 4：安裝 Codeer Agent Skill

使用 Codeer CLI 0.1.17 或更新版本安裝。安裝 Skill 不需要 Codeer API key； 操作 Codeer 伺服器時才需要前面設定的 profile。

Claude Code

```
codeer skill install codeer-agent --target claude
```

Codex

```
codeer skill install codeer-agent --target codex
```

預設安裝到使用者層級：Codex 使用 ~/.agents/skills/codeer-agent， Claude Code 使用 ~/.claude/skills/codeer-agent。加上 --scope project 會改成目前目錄下的 .agents/skills/ 或 .claude/skills/。

使用 codeer skill list 查看安裝狀態。--dry-run 會下載並驗證來源、顯示 安裝目的地，但不寫入安裝檔案；--ref <tag-or-commit> 可固定來源版本， 省略時使用 main。來源版本必須包含 skills/codeer-agent 資料夾。

如果安裝後沒有看到 Skill，請重新啟動 Codex；Claude Code 可執行 /reload-skills。

也可以使用 Codex 內建的 installer：

```
$skill-installer install https://github.com/codeer-ai/codeer-skills/tree/main/skills/codeer-agent
```

這種安裝由原本的 installer 管理，codeer skill 不會接管或移除。

更新與移除 Skill

再次執行 install 就會更新未修改的 CLI 管理套件，不必先移除。 先用 --dry-run 預覽，再安裝 main 的版本；也可用 --ref 選擇指定版本。 Claude Code 請將 --target codex 換成 --target claude；若原本使用 --scope project，以下指令也要加上同一選項。

```
codeer skill install codeer-agent --target codex --dry-run
codeer skill install codeer-agent --target codex
codeer skill install codeer-agent --target codex --ref <tag-or-commit>
```

相同內容會回傳 already_installed，不改動檔案；更新成功會回傳 updated。 預覽會回傳 would_install、would_update 或 already_installed。 更新會先下載並驗證完整新版，再替換舊目錄並清除過期檔案；替換失敗時還原 舊版。若還原也失敗，錯誤訊息會提供保留備份的位置，供手動復原。

內容已修改、手動安裝或透過符號連結管理的目錄會被保留；請先保存自訂內容， 再手動處理。若要移除未修改的 CLI 管理套件：

```
codeer skill uninstall codeer-agent --target codex --dry-run
codeer skill uninstall codeer-agent --target codex
```

移除 CLI 不會一併移除 Skill。

Step 5：驗證 Skill 是否被載入

請在 Codex / Claude Code 中問下面這句。這是在確認 assistant 是否真的知道 Codeer Agent 生命週期，而不是只確認 CLI 能不能連線。

```
Please confirm whether the Codeer Agent Skill is loaded. Explain how you can
help me plan an Agent from KB/files, create behavior cases, analyze production
history, and show me proposed changes before applying them.
```

理想回答應該提到：

- Agent lifecycle
- Behavior Cases
- Production History
- 顯示具體修改預覽並核對授權範圍；已確認的同一批內容可沿用授權，發布另行確認

本機專案工作流程

直接描述想要的 Agent，並提供檔案或連結。例如：「依這些 SOP 建立售後 Agent， 可以查訂單，但退款由人工決定。」coding agent 會先讀資料、說明理解並提出建議， 只詢問意圖不明、來源衝突或新的重要取捨，不要求你重複確認已清楚說過的需求。 它會邊探索邊保存設計與待決事項，方便換對話後接續。你不必學習內部模式名稱、 填寫 Agent 設計 模板，或為了記錄明確指示再核准一次文件。

整體流程是：理解需求 → 設計行為與測試 → 預覽並套用已授權的 Draft 修改 → 讀回與審查 → 測試指定版本 → 視結果診斷修正 → 明確授權後發布。 以下命令可由 coding agent 代為操作；設計決策本身不等於遠端寫入授權。

以下命令已在開發原始碼實作；本次變更尚未發布新的 PyPI 套件。 先檢查 codeer init --help，尚未提供時請使用 開發版安裝。

```
codeer init ./support-agent
cd support-agent
codeer status
codeer validate
```

這三個命令不需要憑證或網路。接手既有 Agent 時，先選擇正確 profile，再執行：

```
codeer agent pull --agent <id>
codeer eval pull
codeer agent diff --local
```

一個本機專案對應一個主要 Agent，不會建立遠端 Project。原始 SOP / PDF 放在 sources/documents/，以 sources/index.md 記錄來源；整理後的 KB 放在 kb/。 coding agent 將已確定的設計決策放在 design/agent-design.md，待確認提案放在 design/agent-design.draft.md。log.md 簡記重要理由、狀態與證據連結。 Prompt 的唯一編輯來源是 agent/instructions.md，其他設定放在 agent/settings.json；穩定 Tester / Case / Rubric 配對放在 evals/design.json。

上述為新專案預設。既有專案沿用 [paths].design 或舊 [paths].contract 指定的檔案； 未指定時，若只有舊 design/behavior-contract.md 或其草稿，便沿用舊位置。 新舊兩組檔案並存時，以 [paths].design 明確選定來源。待確認提案放在選定檔案旁的 <stem>.draft.md，不另建第二份設計。更新 CLI 不會搬移既有檔案；純名稱調整不會 使證據過期，設計內容或其他依賴改變時仍需重新驗證。

直接用日常語言提出需求，coding agent 會選擇 build、cases、audit、apply、eval、improve、publish、observe 八種 Skill 模式，也可以明確指定模式。例如「只診斷這題轉人工失敗」、「套用先前確認的修改並測試， 不要發布」，或「從上週 AI Draft 回饋提出測試候選」。這些是 Skill 模式， 不是 CLI 子命令。既有 Agent 可以從局部診斷開始；匯入現況不等於接受業務規則或品質。

專案套用先取得 --dry-run 預覽，再傳入 --expect <token>。 token 只辨識內容，不代表授權。本機 / 網頁衝突與結果不明都會保留，需先核對再重試。 套用 Draft 不等於發布。每輪測試在 runs/ 保存確切 result IDs；發布會核對有效 語意 audit、已確認驗收範圍與涵蓋該範圍的通過 run，再於 releases/ 保存發布讀回。 缺少工具證據或只有舊結果不能算成功；Judge 分數本身也無法證明操作真的發生。

格式、舊專案接手方式及 API 限制請見 專案契約。原始 History / AI Draft 匯出留在 sources/history/<collection-id>/ 私有目錄；候選測試保留來源，經審查才套用。

Codeer Agent Skill 會怎麼和你協作？

Skill 的角色是把 Agent 改善流程拆成可以討論、可以確認的步驟。它不只是 幫你下指令，也會協助判斷下一步該改什麼。

- 先了解你的目標、資料來源和目前 Agent 狀態，再提出適合的工作順序。
- 需要建立或調整 Agent、KB、Behavior Cases、Rubrics 時，會先說明它準備 做什麼。
- 真正套用修改前，會先讓你預覽會改哪些內容，或用修改前後差異的方式讓你 確認。
- 發布新版 Agent 前，會先確認測試結果、失敗分析和修改內容都已經被看過。

CLI 常見問題快速排除

codeer command not found

執行：

```
pipx ensurepath
```

然後重開 terminal 再試一次。

codeer check 顯示 401 / 403

API key 可能過期、被撤銷或權限不足。請重新建立 Admin workspace key 並更新 profile。

```
codeer profile add work
codeer profile use work
codeer check
```

看到錯的 workspace

目前 profile 使用了另一個 workspace 的 API key。改用正確 profile 後再執行：

```
codeer check
```

找不到 Agent

確認 CODEER_AGENT_ID 是否屬於同一個 workspace；不確定時先 unset 後列出 workspace agents：

```
unset CODEER_AGENT_ID
codeer agent list
```

安裝完成後，下一步做什麼？

如果你還沒有 Agent

可以問：

```
這些是我的 KB / 文件資料夾，請先跟我討論適合建立什麼 Agent v0、需要哪些資料、有哪些風險邊界。
```

如果你有 Agent，但還沒有 Behavior Cases

可以問：

```
請根據這個 Agent 的任務，選擇或設計 Tester、確認必要證據，再提出 Case、Rubric 和配對，先讓我審核再套用。
```

如果你有 Behavior Cases，但有些沒有通過

可以問：

```
請分析失敗原因，判斷應該修 KB、Agent instructions、retrieval，還是 Standard / Rubric，並先讓我看會改哪些內容。
```

如果你的 Agent 已經上線

可以問：

```
請分析 Production History / Negative Feedback，找出常見失敗模式、缺少的測試覆蓋，以及下一輪應優先處理的問題。
```

如果你準備開始下一輪迭代

可以問：

```
請根據目前問題、測試結果與歷史紀錄，提出下一版修改計畫；先讓我看證據與會改哪些內容，再決定是否套用或發布。
```